

DS-1000: A Natural and Reliable Benchmark for Data Science Code Generation



Yuhang Lai¹, Chengxi Li¹, Yiming Wang^{1,2}, Tianyi Zhang³, Ruiqi Zhong⁴, Luke Zettlemoyer^{5,6}, Scott Wen-tau Yih⁶, Daniel Fried⁷, Sida Wang⁶, Tao Yu^{1,5}

¹ The University of Hong Kong ² Peking University ³ Stanford University ⁴ UC Berkeley ⁵ University of Washington ⁶ Meta AI ⁷ Carnegie Mellon University

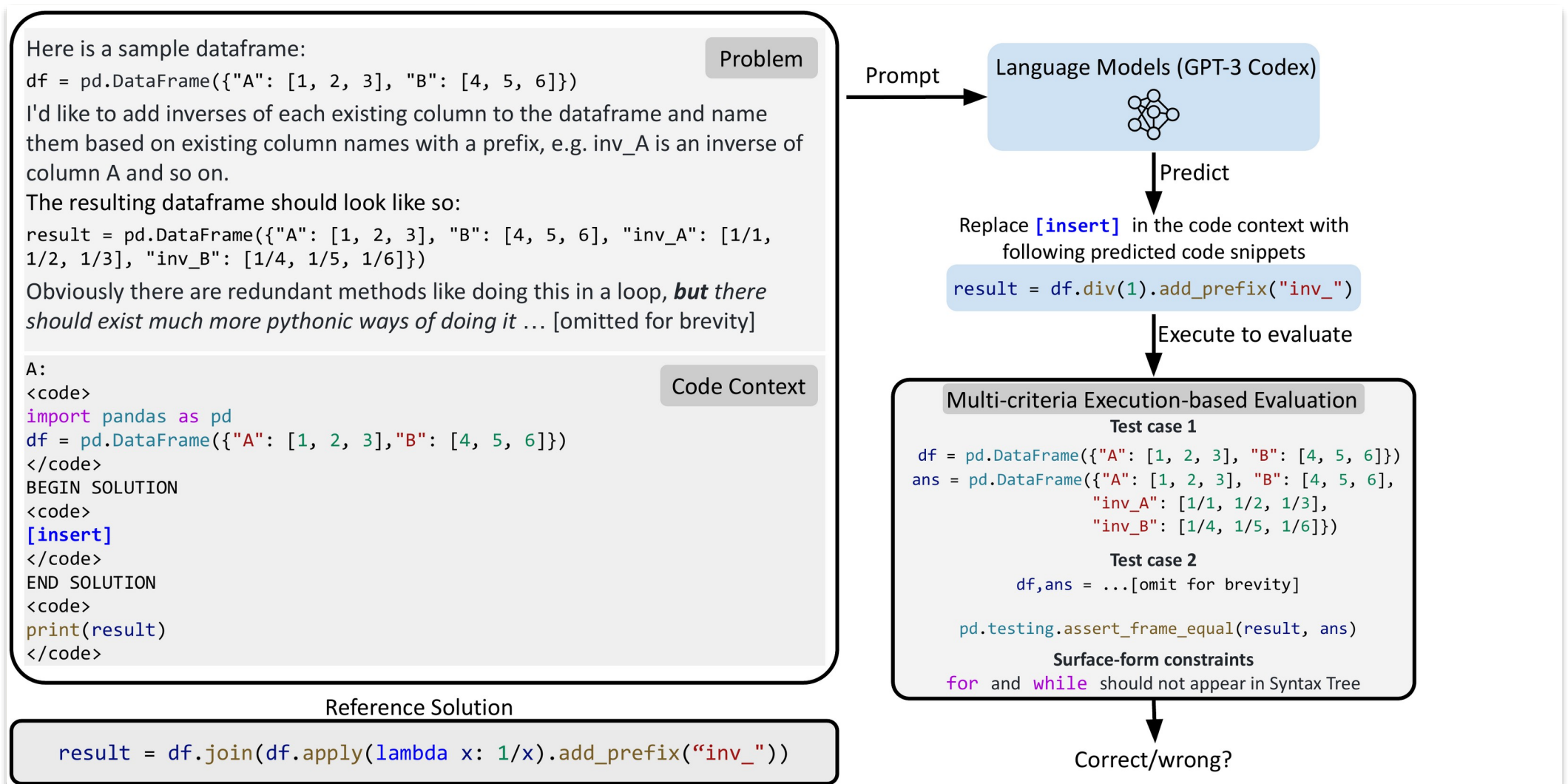


Problem

Existing code benchmarks target competition or interview-style puzzles. They lack naturalistic data science context, and often score answers with unreliable surface metrics like **BLEU**. No prior benchmark combined naturally occurring, diverse-context problems with reliable execution-based evaluation.

Motivation

- Real StackOverflow problems carry rich context — buggy code, errors, I/O examples — that prior benchmarks omit.
- As models improve, surface metrics diverge from intent; execution-based evaluation becomes essential.



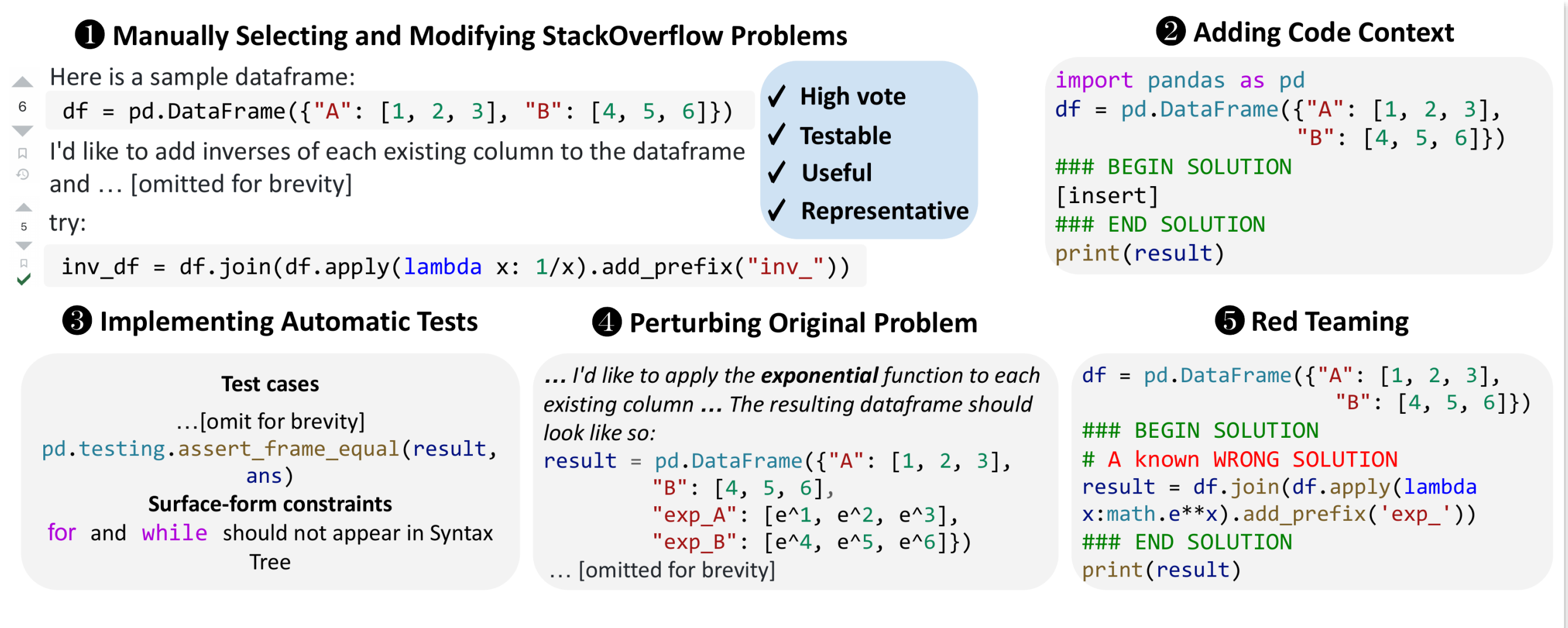
An example DS-1000 problem: the model fills [insert]; the code is executed against test cases and surface-form constraints.

Method

- Select & rewrite** high-vote, testable, useful, representative StackOverflow posts, then **add code context** with insertion markers.
- Implement automatic tests** — execute test cases and check surface-form (API / keyword) constraints in the syntax tree.
- Perturb** each problem to defend against memorization, then **red-team** the evaluation to reject known-wrong solutions.
- Every problem, solution, and metric is reviewed by at least **three expert annotators**; problems use an official **insertion (infilling)** prompt format.

RELIABLE

Accepted solutions must pass executable test cases **and** surface-form API/keyword checks — a multi-criteria bar that rejects plausible-but-wrong code.



The five-stage pipeline for building DS-1000: select & modify, add code context, implement tests, perturb, and red-team.

Dataset / Benchmark

- 1000 problems** from **451 unique** StackOverflow posts; over half are perturbed against memorization (**152** surface, **235** semantic, **162** difficult rewrites).

NumPy

Pandas

TensorFlow

PyTorch

SciPy

Scikit-learn

Matplotlib

Key Results

Model	Completion	Insertion
InCoder-6B	7.4	7.5
Codex-Cushman	18.1	—
Codex-001	20.2	—
Codex-002	39.2	43.3

SO WHAT → *DS-1000 separates model strength (7.4% → 43.3%); Insertion adds +4.1% for Codex-002.*

Ablation Study

PERTURBATION	NUMPY-100	DS-1000
Surface	−21.7	−3.4
Semantic	−48.9	−9.0

SO WHAT → *On memorization-prone numpy-100, Codex-002 collapses after perturbation while DS-1000 barely moves — in 36% of semantic cases the model still returns the original answer.*

Headline Numbers

43.3%

Best model (Codex-002, Insertion)

far from solved — ample headroom

1000 problems · 7 libraries

1.8% false discovery rate

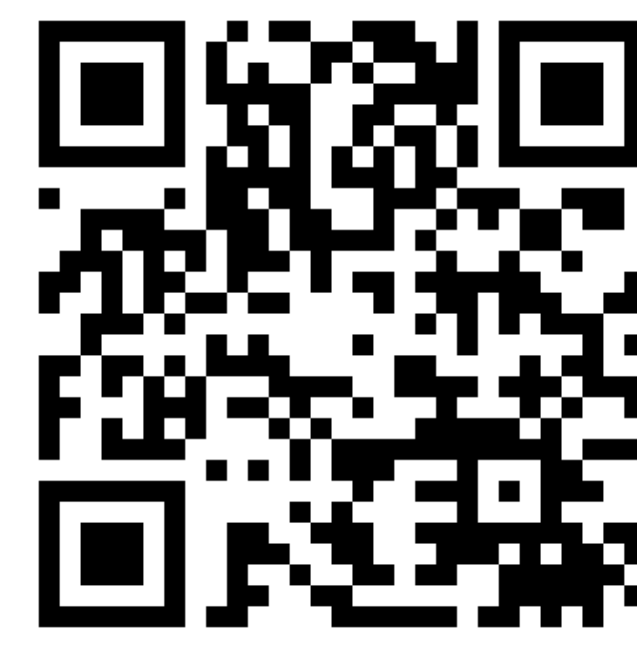
140 avg words / problem

Takeaway

DS-1000 is a realistic, reliably evaluated, memorization-resistant benchmark on which even the best code model reaches only **43.3%** — exposing a large gap in data science code generation. Its execution-based metric and perturbation strategy keep the benchmark trustworthy and genuinely hard to game.

Natural problems + execution-based, multi-criteria evaluation + perturbation = a trustworthy, hard-to-game standard.

Scan to Read



Paper



Code

Project ds1000-code-gen.github.io